Resolución de Laboratorio

Mr. Phisher

Investigación sobre un documento de Word malicioso (.docm) recibido por correo: inspección de la macro VBA incrustada, identificación de la ofuscación mediante XOR con el índice del bucle, replicación de la lógica de descifrado en un script de Python y obtención del flag oculto.

URL: https://tryhackme.com/room/mrphisher

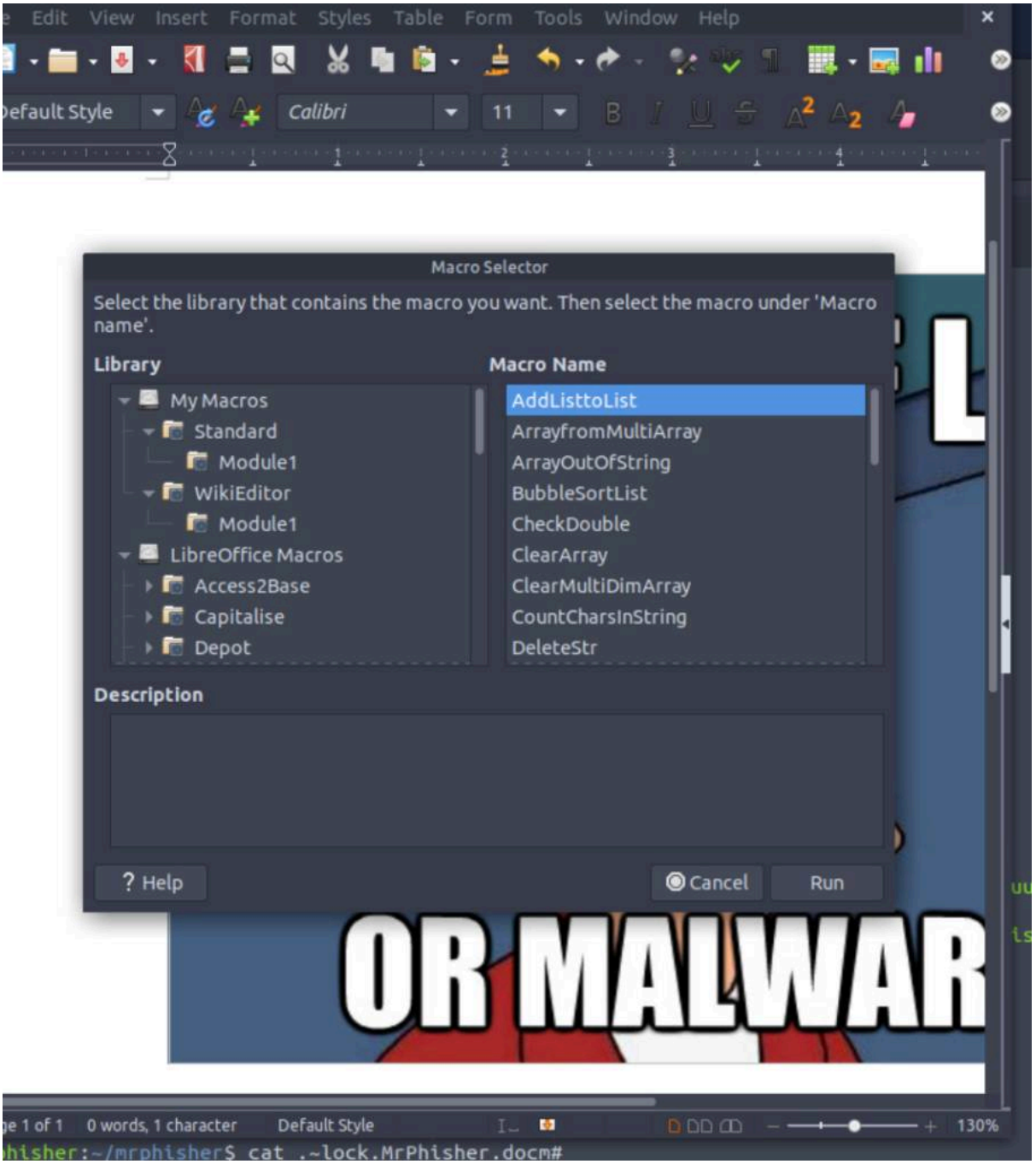
Writeup por Jenrrisinho

Octubre 2026

Fase 1: Análisis Inicial del Archivo

Al iniciar la máquina del laboratorio o descargar el archivo, nos encontramos con un documento de Microsoft Word habilitado para macros: MrPhisher.docm.

Si intentamos abrir el archivo directamente en un entorno Linux utilizando LibreOffice, nos saltará una advertencia de seguridad bloqueando la ejecución automática de macros. Al intentar ejecutar las macros del documento, el programa nos impedirá el paso debido a las políticas restrictivas de la aplicación, mostrando un selector con funciones predeterminadas del sistema en lugar del código dañino.

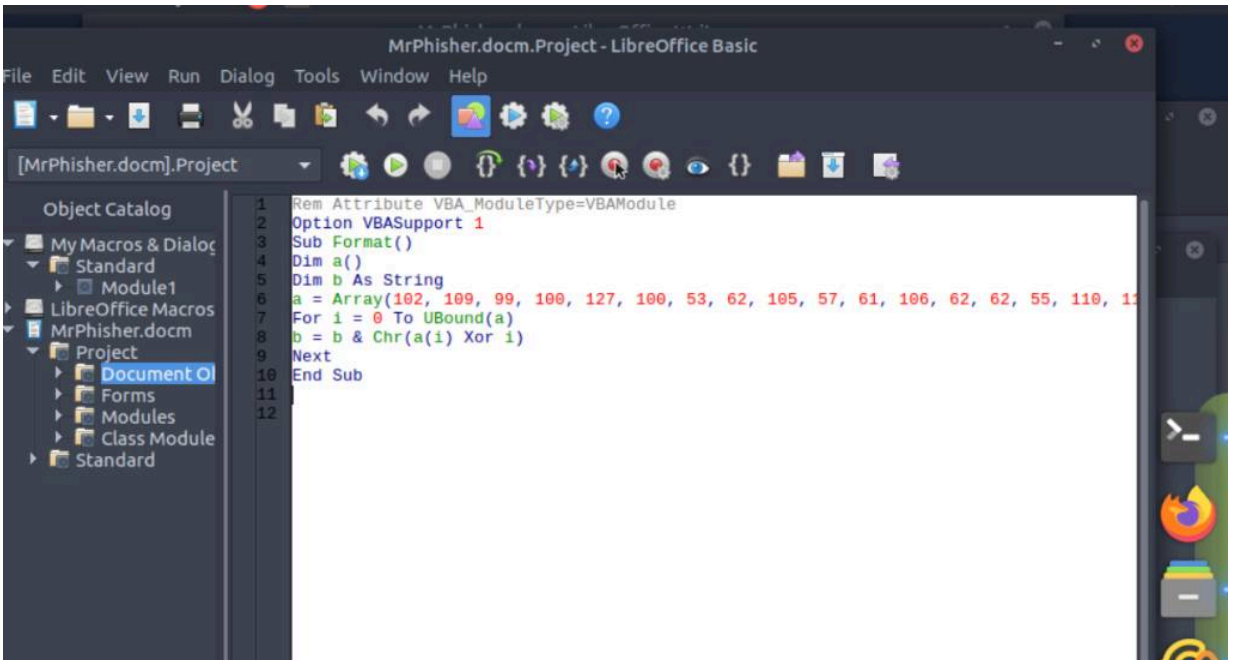


Fase 2: Inspección del Código Fuente (VBA)

Para analizar las tripas del documento sin infectar el sistema, accedemos al editor de código interno presionando Alt + F11 o navegando a través del menú en Tools > Macros > Organize Macros > LibreOffice Basic. Al intentar correr la macro Format desde este menú de organización, la aplicación arrojará un bloqueo inmediato por razones de seguridad.

(Aquí va la captura donde se muestra el cuadro de diálogo Basic Macros con el error For security reasons, you cannot run this macro)

Una vez dentro del editor de código de LibreOffice Basic, localizamos el módulo principal dentro del proyecto de MrPhisher.docm y descubrimos un fragmento de código ofuscado que contiene un arreglo con una lista de números decimales cifrados y un bucle For.



Análisis de la lógica del script:

1. El Arreglo a: Almacena una lista de números decimales que representan los caracteres ocultos del flag. Si intentamos convertirlos directamente a ASCII, obtendremos caracteres ilegibles porque están cifrados.
2. El Bucle For: Recorre el arreglo número por número. La variable i actúa como contador, incrementando de 1 en 1 en cada vuelta empezando desde cero.
3. La Operación Xor i: Esta es la técnica de evasión. El script aplica una operación lógica XOR binaria combinando cada número del arreglo con su posición actual. Al cambiar el valor de la clave en cada iteración, rompe el patrón clásico del cifrado estático y despista a los antivirus tradicionales.
4. La función Chr() y la concatenación &: Traduce el número resultante del XOR a su letra correspondiente y la va pegando consecutivamente en la variable de texto b para que las letras no se borren entre sí y se acumulen en una cadena completa.

Fase 3: Extracción del Flag

Para forzar a la propia macro a que haga el trabajo matemático por nosotros y nos muestre el resultado en una interfaz gráfica, debemos realizar una pequeña modificación en el entorno controlado de desarrollo.

Primero, ingresamos a las opciones de LibreOffice en Tools > Options > LibreOffice > Security > Macro Security y cambiamos temporalmente el nivel de seguridad a Low (Bajo). Tras guardar los cambios, cerramos por completo la suite de oficina y volvemos a abrir el archivo para eludir las restricciones.

De vuelta en el editor de código, insertamos la instrucción MsgBox(b) justo antes de terminar la función, posicionándola entre la palabra Next y la línea End Sub. Al presionar el botón de ejecutar en la barra superior, el programa procesará el algoritmo de descifrado fila por fila y desplegará una pequeña ventana emergente en el centro de la pantalla con el texto final en texto plano.

